FactoryBench: Benchmarking Machine Understanding via Question-Answering

Yanis Merzouki[1,*] **Coral Muniz**[2] **Balázs Günther**[1] **Matei Ignuta-Ciuncanu**[3,4] **Marcos Bracamonte**[1,5]
Alessandro Lombardi[2] **Camilla Mazzoleni**[2] **Federico Martelli**[2,1] **Riccardo Maggioni**[2] **Jonas Petersen**[1,2,†] **Philipp Petersen**[6,†]

[1]ETH Zurich [2]Forgis [3]Imperial College London [4]University of Berkeley [5]KTH [6]University of Vienna
[*]Correspondence: `ymerzouki@ethz.ch` [†]Equal senior contribution

Code: `github.com/Forgis-Research/FactoryBench`
Dataset: `huggingface.co/datasets/forgis/factorybench`

Abstract

We introduce **FactoryBench**, a benchmark for evaluating time-series models and LLMs on machine understanding over academic and industrial time-series data. Q&A pairs are organised along four causal levels (state, intervention, counterfactual, decision) instantiating Pearl's ladder of causation on robotic telemetry, and span five answer formats: four structured formats are scored deterministically and free-form answers are scored by an LLM-as-judge voting protocol. We propose a scalable Q&A generation framework built around structured question templates, present **FactoryWave** (a dense, multitask, multivariate sensor dataset collected from a UR3 cobot and a KUKA KR10 industrial arm at 125 and 83 Hz), and construct FactoryBench as a large-scale benchmark grounded in FactoryWave alongside the AURSAD and voraus-AD open-source datasets. Together, these provide a rigorous testbed for evaluating reasoning, causal understanding, and decision support over industrial signals.

1 Introduction

Modern industrial systems generate large volumes of multivariate time-series data from sensors, actuators, and controllers. In robotic manufacturing settings, these signals encode joint states, torques, forces, velocities, contact events, task phases, and fault indicators. Extracting actionable knowledge from such data is central to monitoring, diagnostics, anomaly detection, and decision support.

Traditional time-series models excel at narrow tasks such as prediction, classification, and anomaly detection Zhou et al. [2021], Su et al. [2019], Audibert et al. [2020]. However, these models are often specialized and difficult to interpret. They typically output labels or numeric predictions without explicit reasoning or higher-level explanations, and generalize poorly outside of the specific task they are trained on Chalapathy and Chawla [2019], Lavin and Ahmad [2015], Zeng et al. [2023], which limits their direct utility for automated engineering decision-making.

Large language models, in contrast, exhibit strong general reasoning abilities and can produce structured explanations in technical contexts Wei et al. [2022], Yao et al. [2023a]. They can integrate textual information, follow logical chains, and generate interpretable outputs. Yet, when applied directly to dense numerical time series, general LLMs still underperform without adaptation Gruver et al. [2023], Jin et al. [2024]. In contrast, specialized transformer-based architectures have become increasingly effective for time-series forecasting and representation learning Vaswani et al. [2017], Zhou et al. [2021], Wu et al. [2021], Zhou et al. [2022], Nie et al. [2023], Ansari et al. [2024], Das et al. [2024].

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Track on Evaluations and Datasets.

A promising paradigm is to build LLM agents equipped with specialized tools, including time-series models and signal-processing modules Schick et al. [2023], Yao et al. [2023b], Qin et al. [2023]. These agents can combine language reasoning with domain-specific computation. Nevertheless, evaluating whether such systems truly reason about machine behavior rather than pattern-matching on surface textual cues remains challenging. By machine-behavior understanding we mean the ability to (i) interpret the current operational state from raw multivariate signals, (ii) predict the effect of an intervention on the system, (iii) reason counterfactually about what would have happened under alternative histories, and (iv) recommend remedial actions grounded in physical and engineering constraints. To the best of our knowledge, how well current LLMs perform across these four competences on dense industrial telemetry has not yet been systematically measured. General reasoning benchmarks such as MMLU, BIG-bench, and MMMU Hendrycks et al. [2021], Srivastava et al. [2023], Yue et al. [2024] target textual, code, or multimodal understanding and are structurally unable to probe machine behavior; benchmarks closer in spirit, such as TimeSeriesExam, ChatTS, EngineMT-QA, TSAQA, Time-MQA, MTBench, and QuAnTS (Table 1), evaluate aspects of time-series Q&A but either restrict themselves to synthetic or univariate data, omit counterfactual reasoning, or do not involve a robotic system under closed-loop control, leaving a gap in rigorous evaluation of machine-behavior reasoning over industrial telemetry.

To address this gap, we propose FactoryBench, a benchmark designed to evaluate machine-behavior reasoning in time series models and LLMs. FactoryBench is grounded in FactoryWave, a dense multivariate time-series dataset collected from one-armed robotic systems spanning both collaborative and industrial manufacturing platforms, with synchronized setpoint, context, feedback, and effort signals recorded at 83 and 125 Hz (KUKA KR10 and UR3 respectively). Industrial-robot data of this kind is rare in public benchmarks: these systems are almost exclusively deployed in production environments with safety enclosures that restrict access, proprietary controllers that expose limited telemetry, and recordings that typically fall under plant-level confidentiality agreements. As a result, existing robotic datasets are dominated by cobot platforms, and to our knowledge FactoryWave is the first extensive anomaly dataset to (partly) cover an industrial robot, explicitly aimed at bridging this cobot-to-industrial-machine gap.

We also introduce a scalable question-generation framework composed of 20 structured templates spanning a wide range of reasoning types and applicable to general machine data flows. These templates enable systematic construction of question-answering tasks that probe state understanding, intervention reasoning, counterfactual analysis, and decision-making in machine contexts. Using this framework and the FactoryWave dataset, we build FactoryBench, a large-scale Q&A dataset specifically designed to evaluate time-series understanding and engineering reasoning in LLM agents.

By bridging the gap between general language reasoning and specialized time-series modeling, FactoryBench provides a principled foundation for studying machine-behavior reasoning in industrial environments.

2 Related work

Recent advances in LLM reasoning include prompting and deliberative strategies (e.g., Chain-of-Thought and Tree-of-Thought) that improve multi-step performance Wei et al. [2022], Yao et al. [2023a], alongside tool-augmented paradigms such as Toolformer and ReAct that enable grounded computation through external modules Schick et al. [2023], Yao et al. [2023b], Qin et al. [2023]. These ideas motivate industrial agent design where symbolic reasoning must be combined with numerical workflows.

Time-series modeling has advanced through transformer-based architectures (Informer, Autoformer, FEDformer, PatchTST) Zhou et al. [2021], Wu et al. [2021], Zhou et al. [2022], Nie et al. [2023], strong alternative baselines such as TFT and N-BEATS Lim et al. [2021], Oreshkin et al. [2020], and critical comparisons against simpler linear models Zeng et al. [2023]. Foundation-model directions (e.g., Chronos) further explore transfer and zero/few-shot adaptation Ansari et al. [2024], Das et al. [2024]. In parallel, anomaly and reliability monitoring literature includes OmniAnomaly, USAD, and Deep SVDD Su et al. [2019], Audibert et al. [2020], Ruff et al. [2018], with surveys and benchmarks highlighting persistent gaps in interpretability and root-cause actionability Chalapathy and Chawla [2019], Lavin and Ahmad [2015].

Causal frameworks provide formal tools for interventions and counterfactuals Pearl [2009], Peters et al. [2017], Rubin [1974], while temporal methods such as Granger causality and modern nonlinear causal discovery support directional reasoning in time series Granger [1969], Runge et al. [2019]. Existing broad benchmarks (MMLU, BIG-bench, MMMU) Hendrycks et al. [2021], Srivastava et al. [2023], Yue et al. [2024] and classical time-series resources Laptev et al. [2015], Dau et al. [2019], Wen et al. [2022] do not directly evaluate machine-centered reasoning over industrial telemetry. FactoryBench targets this gap by jointly testing temporal interpretation, causal reasoning, and engineering decision support. Table 1 positions FactoryBench against closely related Q&A and time-series benchmarks along eight axes. The column labels refer to concepts introduced later in the paper: Counterfactual denotes questions that reason about alternative histories (Level 3, Section 3.1); Free-Form denotes open-ended natural-language answers, scored by an LLM-as-judge voting protocol (Appendix A); and Novel Dense TS indicates whether the benchmark contributes a new, high-frequency multivariate dataset rather than relying exclusively on public sources.

Table 1: Comparison of FactoryBench with existing benchmarks. "Counterfactual" = questions reasoning over alternative histories; "Free-Form" = open-ended answers judged by LLMs; "Novel Dense TS" = contributes a new multivariate high-frequency dataset.

Benchmark	Machine/Robot	Multivariate TS	Size	Counterfactual	Ranking	Numerical	Free-Form	Novel Dense TS
TimeSeriesExam	×	×	∼700	×	✓	×	×	×
ChatTS	×	✓	∼134k	×	✓	✓	×	×
EngineMT-QA	✓	✓	∼110k	×	✓	✓	✓	×
TSAQA	Partial	×	∼210k	×	✓	×	×	×
Time-MQA	×	✓	∼200k	×	✓	✓	✓	×
MTBench	×	✓	?	×	✓	✓	✓	×
QuAnTS	×	✓	∼150k	×	✓	✓	✓	×
CLEVR	×	×	∼1M	×	×	✓	×	×
CLEVRER	×	×	∼305k	✓	×	✓	✓	×
FactoryBench (Ours)	✓	✓	**TBD**	✓	✓	✓	✓	✓

3 FactoryBench framework

3.1 Four levels of machine understanding

Table 2: FactoryBench levels, what they test, example questions, and expected answer formats.

Level	Example question	Type	What it tests	Answer format
1	"We want to isolate the lifting phase in the robot's time series. Assuming a fixed window length of 15 timesteps, at which timestamp should the window begin?"	State	Can the model understand what the machine is doing and how it is behaving?	Tensor
2	"A collision with a foam cube occurs at $T{=}850$ ms. Rank signal segments (A–D) in the order you would expect them to appear after the event."	Intervention	Can the model reason about how the machine will behave in the face of an event?	Ranking
3	"Had a payload misconfiguration occurred at $T{=}200$ ms, what would the target torque on joint 2 at $T{+}50$ ms have been in this counterfactual case?"	Counterfactual	Can the model reason accurately about theoretical scenarios?	Scalar
4	"Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the root cause and the steps to fix it."	Decision	Can the model make informed decisions about the machine?	Free-form

Our four-tier hierarchy is explicitly built on top of Pearl's ladder of causation association, intervention, and counterfactual reasoning which has become the organizing principle for causal inference and is increasingly adopted as an evaluation axis for machine-learning systems Pearl [2009], Peters et al. [2017], Jin et al. [2023], Pawlowski et al. [2020], Peyrard and West [2020]. We extend the three causal rungs with a fourth decision-making rung that reflects how industrial robotic platforms are actually operated: after interpreting state and causal relationships, operators must select and execute corrective procedures, typically prescribed by vendor manuals (e.g., Universal Robots error-code handbooks). This final rung aligns with the diagnostic-then-act loop that is standard in fault-tolerant control. Grounding the hierarchy in these established principles ensures that each level probes a qualitatively distinct reasoning skill and that failure modes are interpretable in terms of the underlying causal or decision-theoretic primitive.

To systematically evaluate machine-behavior reasoning, we organize question-answering tasks according to this four-tier hierarchy, each probing distinct reasoning capabilities:

Level 1: State. This level assesses the agent's ability to interpret the current state of the machine under normal conditions, including identification of operational modes, prediction and recognition of sensor patterns. Questions at this level require accurate extraction and interpretation of time series features.

Level 2: Intervention. At this level, the agent must reason about the consequences of interventions or events occurring at the present timestep that might perturb the distribution of machine states. Tasks include predicting the immediate impact of control actions, diagnosing faults as they arise, and understanding causal relationships in real time for both normal and anomalous episodes.

Level 3: Counterfactual. This level evaluates the agent's ability to reason about hypothetical scenarios, such as the effect of an event or intervention at a previous timestep. Questions require the agent to simulate alternative histories and assess how outcomes would differ under counterfactual conditions, while still considering the history they know.

Level 4: Decision Making. The highest level evaluates the agent's ability to act on its understanding of the system. Given a recorded episode and a task description, the agent must produce a sequence of actions or recommendations to answer that prompt. In FactoryBench, this targets troubleshooting and optimization, and combines the skills exercised at the previous three levels: reading the system state, reasoning about causes, and weighing alternative interventions. It reflects what we expect from an LLM acting as an engineering assistant on the factory floor.

By structuring Q&A tasks along these four levels, FactoryBench enables rigorous and granular assessment of machine-behavior reasoning, from basic state recognition to advanced decision support. Figure 1 instantiates Pearl's three causal rungs (L1–L3) on robotic time-series data and adds the L4 decision-making layer that FactoryBench introduces on top. Each question is emitted in one of five answer formats (single-select MCQ, multi-select MCQ, ranking, tensor, free-form); the four structured formats are scored deterministically and free-form answers are scored by an LLM-as-judge voting protocol, with per-format details in Appendix A.

3.2 Time series data and causal schema (SCE)

So that the dataset structure itself encodes causality, all time-series signals are organized into three causal groups: **Setpoint** (the controller's commanded target), **Context** (physical and configured conditions, split into static episode metadata and dynamic time-series channels), and **Effort+Feedback** (the machine's measured response). Under healthy operation the response is a known function of setpoint and context; faults manifest as structured residuals from that baseline, giving a single operational fault definition that applies across machines and tasks. Full per-group channel mappings, the formal residual definition, and per-robot calibration details are deferred to Appendix H.

The data conforms to this schema across two source types:

- **FactoryWave:** A custom dataset generated from one-arm robotic platforms executing canonical industrial tasks (e.g., pick-and-place) under varied conditions and systematically injected anomalies.
- **Open Source Datasets:** Adapted datasets such as AURSAD and voraus-AD, offering diverse industrial scenarios and preprocessed to conform to the unified episode structure.

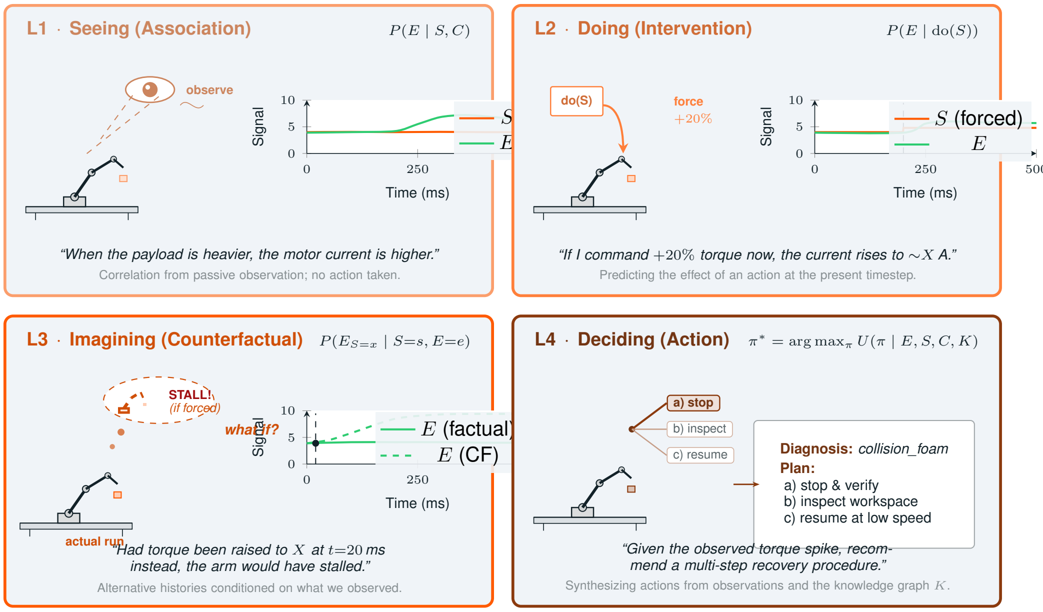

Figure 1: Pearl’s three rungs of causal reasoning (L1–L3), instantiated on robotic time-series, plus the FactoryBench L4 decision-making layer. **L1 (Seeing)** captures correlations from passive observation; **L2 (Doing)** predicts the effect of an intervention; **L3 (Imagining)** reasons over alternative histories conditioned on what was observed; **L4 (Deciding)** synthesizes a recovery procedure by combining the observed signals with manufacturer manuals and the FactoryBench knowledge graph K.

The datasets integrated into FactoryBench are summarized in Table 3; all provide the full setpoint–context–effort triplet at 83 Hz or higher and have been (re)labelled to conform to our unified episode schema.

Table 3: Datasets incorporated into FactoryBench. FactoryWave is collected and released with this work; the other two are open-source datasets adapted to our schema. Tasks: PnP = pick-and-place, Scr = screwing, PiH = peg-in-hole.

Dataset	Robot	Episodes	Frequency	Tasks	Anomalies
AURSAD	UR3e	4094	100 Hz	Scr	4
voraus-AD	Yu-Cobot	2122	100/500 Hz	PnP	12
FactoryWave (ours)	UR3, KUKA KR10	8983	125/83 Hz	PnP, Scr, PiH	27

4 Reliable Q&A generation

4.1 At scale generation via extensive labelling

Scalable ground-truth generation is the central challenge of any Q&A benchmark grounded in raw data. FactoryBench addresses this by coupling a structured labelling ontology with a context-free grammar (CFG)-style template system, designed by robotics experts and PhD researchers. Each template contains variable slots filled at generation time from one of two sources: label information read directly from the episode data (signal names, timestamps, anomaly labels, root causes), or a predefined sampling distribution attached to that slot (prediction horizons, numerical thresholds, curated vocabularies for discrete options). The discrete vocabularies are seeded from the label sets of AURSAD and voraus-AD, extended to cover FactoryWave-specific cases, and released in full so every instantiation is inspectable and reproducible. The context time series used to fill the variables is sampled uniformly by data source (open source vs FactoryWave), then dataset, experiment, length within controlled bounds, and finally placement, yielding a combinatorial expansion from a small set of carefully designed templates into a large, diverse question pool.

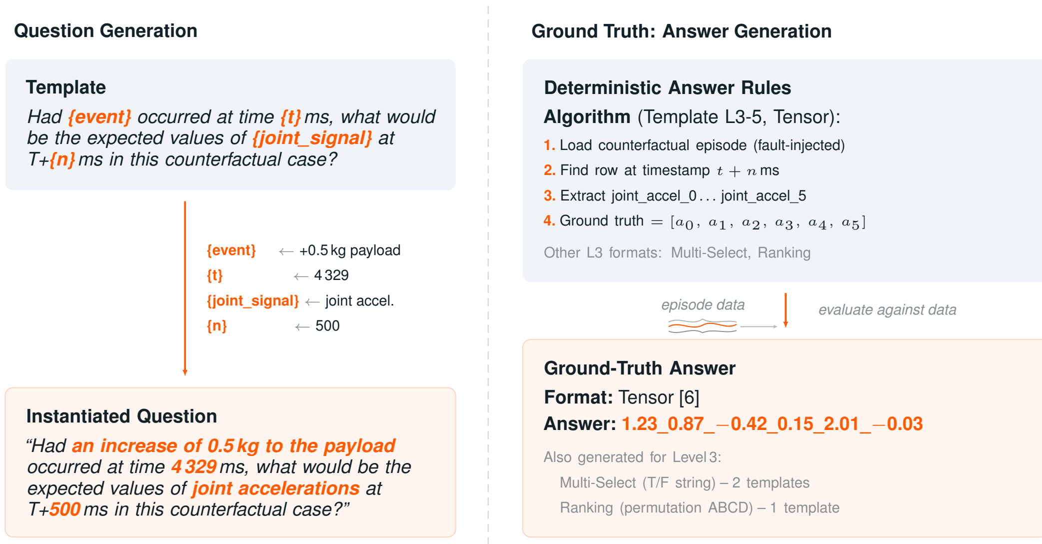

Figure 2: Question-and-answer generation pipeline (Level 3 Template 5 shown; full inventory in Table 6 and Appendix F). **Left:** Typed placeholder slots are bound to episode-specific values. **Right:** Ground-truth answers are computed deterministically from paired episode data via template-specific rules.

Each of the 20 templates is manually authored to probe one specific machine-understanding level while remaining general enough to admit a wide range of instantiations across episode segments, signal names, event descriptions, timestamps, and predicted values. Answer options for single- and multi-select questions are drawn from a shared pool of pre-defined verifiable statements, each paired with a rule that can be evaluated deterministically against the time series given the densely labelled data, so ground truth is never imputed or inferred but computed directly from the underlying labels. This is also our primary defence against the natural concern that template-generated benchmarks can be solved by learning template surface structure rather than the underlying reasoning: every template admits a combinatorially large instantiation space over signal names, timestamps, prediction horizons, payload conditions, and injected-fault types, so two questions sharing the same template rarely overlap in more than a fraction of their variable slots, and the correct answer is never recoverable from the question text alone since it is always determined by the paired time series, which varies across instantiations.

4.2 Density of FactoryWave

FactoryWave is collected from two physical robots (UR3 and the KUKA KR10) executing up to three canonical industrial tasks each: pick-and-place, screwing (UR3 only), and peg-in-hole. Each complete task cycle constitutes one episode, recorded at 125 and 83 Hz across sensor channels covering joint setpoints, feedback, torques, speeds, estimated contact forces, TCP pose, gripper state, and task-phase labels.

find number

Episodes are organized into three experiment types. **Normal** episodes capture nominal operation across three payload levels (light, medium, heavy for pick-and-place). **Fault** episodes inject one of 27 fault types (spanning gripper failures, payload misconfigurations, collisions, and peg-in-hole-specific anomalies), each recorded with fault-specific metadata and, where applicable, a precise injection timestep. **Counterfactual** episodes provide ground truth for Level 3 causal reasoning by approximating the do-operation on physical hardware, via the protocol described next.

Counterfactual generation approximates the last step of Pearl's ladder on physical hardware, which cannot reproduce a bit-identical pre-injection state. For each baseline we re-execute the task 3–5 times with every controllable condition held fixed (object pose, payload, controller configuration, exact scripted trajectory) and inject the target fault at a fixed timestep $t^{\star}$. We then keep the run whose pre-injection segment minimises the signature kernel MMD against the baseline, and use that episode as an approximate groundtruth for hypothetical divergence of baseline.

Table 4 summarises the resulting distribution of episodes by robot, task, and condition. **Counterfactual** entries count CF *groups*: each group is stored as a single paired episode bundling the baseline run with its retained fault counterpart.

Table 4: Distribution of FactoryWave episodes by robot, task, and condition. Counterfactual entries count CF groups (one baseline plus one retained fault run per group).

Robot	Task	Normal	Fault	CF groups	Total
UR3	Pick-and-Place	1053	3238	109	4400
UR3	Screwing	230	850	40	1120
UR3	Peg-in-Hole	322	1200	100	1622
KUKA KR10	Pick-and-Place	578	1221	42	1841
Total		**2183**	**6509**	**291**	**8983**

4.3 Knowledge graph and protocol extraction

Reliable ground truth for Level 4 troubleshooting questions requires more than episode labels: it demands machine-specific recovery protocols grounded in manufacturer documentation. For each robot in FactoryBench we parse the manufacturer's official runtime error documentation into a structured mapping from error codes to error names, descriptions, and recommended recovery steps, capturing what the controller reports when a fault occurs and what an operator should do to resolve it. We then map every anomaly type studied in FactoryBench to the most likely corresponding manufacturer error code, with PhD-level robotics experts reasoning about the physical cause of each anomaly and identifying which runtime error it would most plausibly trigger on the target robot. Where a physical fault injection (such as gripper misactivation, peg misalignment, or external collision) has no well-defined controller error, the same experts author the recovery protocol directly using the same structure for consistency.

The complete mapping (error-derived and expert-authored protocols alike) is ingested into a knowledge graph alongside robot specifications, task definitions, and signal schema. At question generation time, the pipeline queries this graph to automatically assemble the relevant protocol context and inject it as ground truth into Level 4 troubleshooting Q&A pairs, without requiring per-question human review.

5 Experiments

5.1 Zero-shot evaluation

We evaluate a cross-vendor panel of frontier LLMs available at submission time (April 2026): Claude Sonnet 4.6 Anthropic [2025] and GPT-5.1 OpenAI [2025] on the closed-weight side, and DeepSeek V3.2 DeepSeek-AI [2025], Mistral Large 3 Mistral AI [2025], and Qwen3-235B Qwen Team, Alibaba Group [2025] on the open-weight side. We also include Qwen3-4B Qwen Team, Alibaba Group [2025] in zero-shot mode as a lightweight open-weight reference point; its finetuned variant is the subject of the experiment described in Section 5.2. For calibration we report a non-LLM baseline that dispatches per item by answer format: linear regression on the target signal for scalar and tensor templates, and uniform random over the option set for single- and multi-select MCQ and ranking templates. Free-form Level 4 templates are excluded from the baseline (no traditional method maps cleanly to producing a remediation protocol).

Figure 3 reports chance-corrected accuracy for the full panel.

Three patterns stand out. First, on Levels 1–3 the panel stratifies cleanly by model scale and capability: Claude Sonnet 4.6 leads across all three levels (46.8%, 47.1%, 45.9%), Qwen3-235B is the strongest open-weight competitor especially on L3 (43.6%), and Mistral Large 3 follows (34.6%, 31.7%, 36.3%). The two weakest models, Qwen3-4B (21.8%, 27.5%, 28.8%) and DeepSeek V3.2 (25.0%, 29.1%, 28.5%), are only marginally above or below the simple baseline on L1 and L2, indicating that sub-frontier models do not reliably exploit time-series context on state and intervention questions.

Second, Level 4 produces a complete rank reversal. Every model collapses into the 3–15% range, but crucially the ordering reshuffles: GPT-5.1, mediocre on L1–L3 (30.9%, 30.0%, 31.7%), uniquely

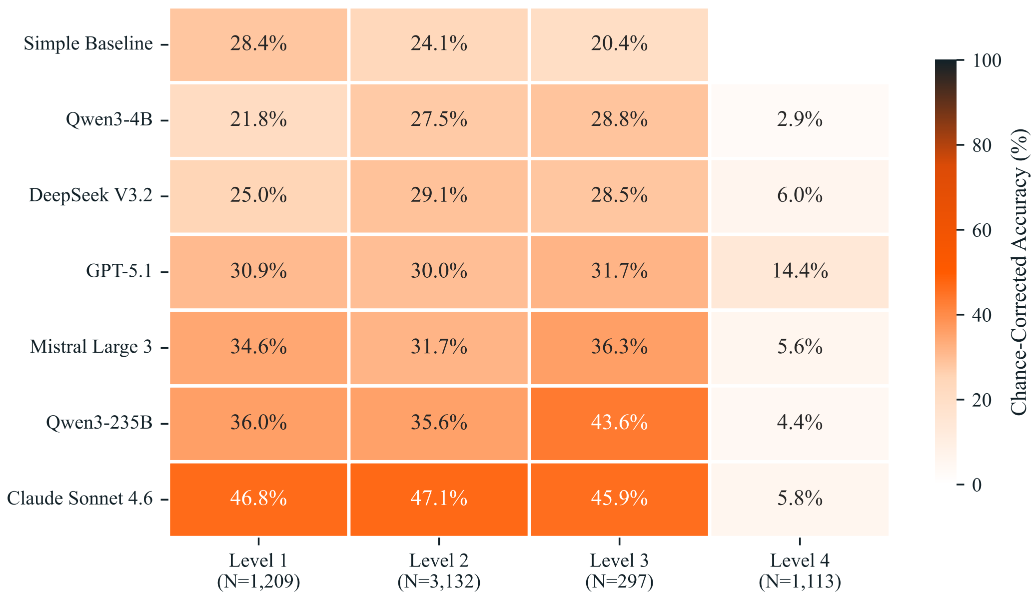

Figure 3: Chance-corrected accuracy (%) on FactoryBench's four reasoning levels. Models are ordered by mean L1–L3 score (ascending). Cell colour encodes absolute accuracy on a shared scale; the Simple Baseline has no L4 entry because free-form templates require language generation. N values on the x-axis report the number of Q&A items per level in the test split.

leads on L4 at 14.4%, while Claude Sonnet 4.6—dominant on L1–L3—scores only 5.8% on L4, statistically indistinguishable from the open-weight small models. The L4 task requires producing a multi-step remediation procedure grounded in the knowledge graph; the rank reversal suggests that L4 probes a qualitatively different capability than signal-reading, one that correlates poorly with L1–L3 performance.

Third, the simple baseline remains competitive: it outperforms Qwen3-4B on L1 (28.4% vs 21.8%) and comes within a few points of DeepSeek V3.2 on all three structured levels. This confirms that the benchmark is not trivially solved by pattern-matching on surface textual cues, and that genuine time-series reasoning is required to separate from chance.

5.2 Industrial generalization

Beyond the zero-shot panel, we run a targeted finetuning experiment to probe whether machine-behavior understanding can be learned from FactoryBench rather than only inherited from a model's pretraining priors, and whether what is learned transfers across machine families. We finetune Qwen3-4B on the subset of FactoryBench Q&A pairs grounded in cobot episodes: the UR3 from FactoryWave, the UR3e from AURSAD, and the Yu-Cobot from voraus-AD. FactoryBench is partitioned $80/10/10$ into train, validation, and test at the episode level (the same mapping is shared across all four levels, so no episode appears in different splits in different levels). The training and validation splits cover Levels 1–3 only and span every answer format used at those levels; no episodes from industrial-grade hardware enter training. We deliberately exclude Level 4 from train and validation because the L4 troubleshooting and optimization templates draw their root causes and remediation protocols from a closed catalogue (the FactoryBench knowledge graph), so training on them would collapse the task into a small classification problem rather than incentivising the model to reason. We are instead interested in whether competence on Levels 1–3 transfers to L4 zero-shot, which is the regime that matters in practice: an LLM acting as an engineering assistant on the factory floor is graded on whether it produces the right diagnosis and remediation procedure, not on whether it can recall a label seen in training.

We then evaluate the finetuned checkpoint on two held-out test sets. The first is the common test split used for the zero-shot panel in Section 5.1, so the finetuned model can be compared item-for-item against the frontier baselines on the same questions. The second is a KUKA-specific held-out split composed exclusively of Q&A pairs grounded in industrial-robot episodes (the 1,841 KUKA KR10 episodes from FactoryWave plus the heavier-payload industrial platforms in the simulation pool), which contains no episodes seen during training and probes the cobot-to-industrial transfer claim directly. The second split is what makes the experiment operationally useful for industrial AI: cobots are cheap and safe to instrument, vary, and inject faults into, while production industrial robots, with their high payloads, high speeds, and stricter safety-stop interlocks, are not. A positive transfer result, where the finetuned model's score on the KUKA split is close to or above the zero-shot panel, would indicate that the diagnostic and counterfactual reasoning skills FactoryBench measures can be acquired cheaply on collaborative platforms and deployed on production hardware. A negative result would localise where machine-behavior reasoning is platform-specific and quantify the cobot-to-industrial gap, which is itself a useful artefact for future benchmarks and sim-to-real training pipelines.

5.3 Sim2real gap analysis on counterfactual pick-and-place

Level 3 questions require comparing what *did* happen against what *would have* happened under a different intervention, holding the rest of the world fixed. Physical executions cannot reproduce identical initial conditions, micro-disturbances, or contact dynamics, so a physical "counterfactual pair" is only an approximation of the intended do-operation; simulation, by contrast, can deterministically fork its state and yield bit-identical pre-intervention trajectories. Simulation buys exact counterfactual fidelity at the cost of realism (contact, noise, latency, and manufacturing variability all drift from the physical system) so we treat the two sources as complementary.

The FactoryWave counterfactual protocol (signature-kernel-MMD selection over 3–5 fault repetitions of a fixed-condition baseline) is described in Section 4.2 and Appendix E. In IsaacSim we reproduce the same pick-and-place task with a URDF-matched UR3 and matching workspace; for each episode we run the baseline to completion, then re-run from a saved simulator state at the injection timestep with the fault applicator enabled, so the two branches are bit-identical pre-intervention by construction and require no MMD selection. This lets us generate arbitrarily many counterfactual pairs at negligible cost, including for faults that are too disruptive or unsafe to stage repeatedly on hardware.

Fill in concrete numbers, plots, and per-template gap tables once the IsaacSim pipeline is frozen; see Appendix G.

6 Discussion

6.1 What the results reveal

The heatmap in Figure 3 surfaces three findings that are individually surprising and collectively raise a deeper question about what current models actually measure when they succeed or fail on industrial Q&A.

Level 1 is the hardest level for most models to beat, including the largest one. The simple baseline scores 28.4% on Level 1 after chance correction — higher than on any other level — because L1 is dominated by state-identification questions where a local-window linear extrapolation over the target signal already resolves many items. Against this unexpectedly strong floor, four of the six LLMs fail to separate meaningfully: Qwen3-4B (21.8%) and DeepSeek V3.2 (25.0%) fall below the baseline outright, and GPT-5.1 (30.9%) exceeds it by only 2.5 percentage points, a margin that is negligible relative to the variability of the task. Only Mistral Large 3 (34.6%, +6.2pp), Qwen3-235B (36.0%, +7.6pp), and Claude Sonnet 4.6 (46.8%, +18.4pp) establish a clear separation. GPT-5.1's near-baseline performance on L1 is particularly striking because it is estimated — by parameter-count extrapolation from public benchmarks and inference-cost proxies — to be by far the largest model in the panel. That the largest model is fourth out of six on the most basic reasoning level indicates that raw scale does not reliably confer the ability to extract precise state information from dense industrial signals, and that L1 is measuring something general-purpose pretraining does not directly optimise for.

Models that fail on Level 1 still exceed the baseline on Levels 2 and 3. The L1 rankings would lead one to predict that Qwen3-4B and DeepSeek V3.2, having scored below a non-LLM baseline on state identification, would perform similarly or worse on strictly harder levels. The heatmap shows the opposite: both models clearly exceed the simple baseline on L2 (Qwen3-4B 27.5% vs. 24.1%; DeepSeek V3.2 29.1% vs. 24.1%) and L3 (Qwen3-4B 28.8% vs. 20.4%; DeepSeek V3.2 28.5% vs. 20.4%) despite their L1 deficits. GPT-5.1 follows the same pattern: barely distinguishable from the baseline on L1 (+2.5pp), yet comfortably above it on L2 and L3 (+5.9pp and +11.3pp respectively). The disconnect is structural across the full panel and not an artefact of one model. A plausible interpretation is that L2 and L3 templates are primarily relational and directional — they ask whether a signal increases under an intervention, or whether two trajectories diverge after an event — and LLMs possess this kind of comparative reasoning as a latent competence that does not depend on the precision numerical reading required at L1. Level 1 is therefore not simply "easier" than L2 and L3; it probes a different and apparently harder sub-skill: exact value extraction from a raw multivariate context, which large language models appear to acquire more slowly than abstract causal relational judgment.

GPT-5.1 leads on the most practically relevant level despite underperforming on the others. The most consequential finding in the heatmap is the L4 profile of GPT-5.1. Ranked fourth out of six LLMs on Levels 1–3 (30.9%, 30.0%, 31.7%), it nevertheless leads on Level 4 by a margin that dwarfs the differences between models on the structured levels: 14.4% versus 6.0% for the second-place model (DeepSeek V3.2), 5.8% for Claude Sonnet 4.6, and 5.6% for Mistral Large 3. The absolute advantage of 8.4 percentage points over the next-best model on L4 is comparable in scale to the gap separating first from last across the entire L1–L3 panel. L4 is the level most closely mirroring real factory operations: the model receives a time-series context, must identify the root cause, and must produce a multi-step remediation procedure grounded in the manufacturer documentation injected through the knowledge graph. GPT-5.1's reversal of fortune on this level suggests that its pretraining has accumulated disproportionate exposure to structured industrial documentation — maintenance logs, error-code manuals, troubleshooting runbooks — that lets it match the correct protocol to a fault description even when its underlying signal comprehension, as measured by L1–L3, is comparatively weak. More broadly, the pattern points to a phenomenon that extends beyond GPT-5.1 specifically: models can produce the correct high-level answer to an engineering question while operating with an incomplete understanding of the physical system that generated the evidence. This dissociation between *signal comprehension* and *protocol retrieval* is not unique to FactoryBench, but the benchmark's hierarchical structure makes it visible in a controlled way. It has a direct practical implication: optimising a model for signal-level performance (L1–L3) may not translate into the procedural competence an operator actually needs (L4), and vice versa — the two capabilities likely require different training signal and different evaluation criteria even within the same industrial domain.

Level 4 scores collapse absolutely, revealing an industrial readiness gap. Setting aside the relative ordering within the L4 column, the absolute scores are alarming in their own right. Even GPT-5.1, the L4 leader, reaches only 14.4% on a chance-corrected scale where zero corresponds to pure-chance performance. Five of the remaining six models cluster between 2.9% and 6.0%, meaning their effective performance on Level 4 is statistically indistinguishable from random output after accounting for the number of answer options. This is not the same collapse as a low score on a hard NLP benchmark: the tasks at Level 4 — identifying the root cause of a fault from a labelled time-series context and producing the corresponding multi-step recovery procedure from a knowledge graph provided in the system prompt — are precisely what a factory AI assistant would be asked to do in practice, and the manufacturer documentation required to answer correctly is given to every model in the prompt. The failure is therefore not one of missing knowledge; it is a failure to reliably exploit structured technical information that is directly available, under realistic conditions. Taken together, the L4 results constitute a clear empirical statement: current frontier language models, including the largest and most capable available at the time of writing, are not yet ready for autonomous deployment in factory-level troubleshooting and optimisation roles. They can assist a skilled operator in answering state and intervention questions (Levels 1–3 show meaningful separation from chance for most models), but the moment the task requires generating a grounded, actionable remediation procedure, performance collapses to near-random for nearly all models. Closing this gap is the central challenge FactoryBench is designed to measure.

6.2 Modular and extensible by design

FactoryBench is built to grow. The Q&A generator decouples reasoning templates from the underlying robot, task, and signal schema: every template is parameterised over generic primitives (signal names, phase indices, anomaly labels, fault categories) rather than hard-coded to a particular machine or task, so adding a new platform only requires populating the corresponding entries in the labelling ontology and the per-robot signal mapping. Once those are in place every existing template instantiates against the new platform automatically, and the answer-format scorers, the knowledge graph, and the LLM-as-judge prompts all carry over. As the project evolves we plan to incorporate additional robots and machine families (industrial arms, mobile manipulators, CNC and additive-manufacturing platforms), additional tasks (assembly variants, polishing, inspection), additional gripper and end-effector types, and additional question templates targeting under-represented reasoning patterns. Each addition slots into the existing schema without breaking prior versions, and the versioned release track ensures that every reported number remains pinned to a fixed dataset state while the benchmark itself continues to grow.

6.3 Limitations

FactoryBench has two substantive limitations that future work must address rather than gloss over. First, Level 3 *counterfactual* ground truth in the physical lab is fundamentally approximate: two real runs cannot share an identical pre-injection state, so our signature-kernel-MMD-selected CF pair is only the closest empirical proxy for the do-operation, not the do-operation itself. L3 scores therefore conflate the model's counterfactual reasoning with the quality of that proxy, and simulation closes the gap only at the cost of physical realism. Second, the Q&A pairs are built on a simplified representation of how machine faults actually appear in production, since it is the first benchmark of its sort. Real industrial faults are typically compound (multiple co-occurring root causes), gradual (slow drift over shifts or weeks), and detected through aggregate KPIs rather than single-episode signals; ours are atomic, fast, and drawn from a closed catalogue of 27 physically injected mechanisms, which makes FactoryBench a measurement of in-distribution fault *recognition* rather than the fault *detection* problem operators actually face.

7 Conclusion

FactoryBench introduces a systematic approach to evaluating machine understanding via Q&A. By focusing deeply on one machine family and providing reliable answer generation at scale, we enable rigorous comparison of methods across four levels of reasoning from basic state identification to procedure generation grounded in technical manuals.

Our physical validation protocol closes the loop from benchmark to reality, addressing the fundamental question: can AI systems that excel at industrial Q&A actually help in the real world?

7.1 License

FactoryBench and FactoryWave are released under the MIT License and are freely available for academic and commercial use. Both the episode data and the benchmark artefacts (question templates, paraphrase banks, LLM-as-judge prompts, generator source code, and the full Q&A dataset) are distributed via the public Hugging Face repositories `Forgis/FactoryWave` and `Forgis/FactoryBench_QA_pairs`. The data are fully public; there is no private holdout. The train/validation/test split (80/10/10 at the episode level, shared across all four levels) is described in Section 5.2 and the mapping is included in the release so that evaluees can reproduce the exact partition used in this paper.

We provide a versioned release track (e.g., `v1.0`, `v1.1`) so that reported numbers always refer to a fixed benchmark state; corrections to labels or templates are published as minor-version updates with a public changelog, and the exact version used in any evaluation is stamped into every result file. Bug reports and label-correction requests are tracked on the public GitHub repository.

References

Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024. arXiv:2403.07815.

Anthropic. Claude Sonnet 4.6 model card, 2025. Anthropic model card. `https://www.anthropic.com/claude/sonnet`.

Julien Audibert, Pietro Michiardi, Frédéric Guyard, Sébastien Marti, and Maria A. Zuluaga. USAD: Unsupervised anomaly detection on multivariate time series. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2020.

Raghavendra Chalapathy and Sanjay Chawla. Deep learning for anomaly detection: A survey. *arXiv preprint arXiv:1901.03407*, 2019.

Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024. arXiv:2310.10688.

Hoang Anh Dau et al. The UCR time series classification archive. *IEEE/CAA Journal of Automatica Sinica*, 2019.

DeepSeek-AI. DeepSeek-V3.2 technical report, 2025. DeepSeek-AI technical report.

Clive W. J. Granger. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 37(3):424–438, 1969.

Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. Large language models are zero-shot time series forecasters. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y. Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. Time-LLM: Time series forecasting by reprogramming large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

Zhijing Jin, Yuen Chen, Felix Leeb, Luigi Gresele, Ojasv Kamal, Zhiheng Lyu, Kevin Blin, Fernando Gonzalez Adauto, Max Kleiman-Weiner, Mrinmaya Sachan, and Bernhard Schölkopf. CLadder: A benchmark to assess causal reasoning capabilities of language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Nikolay Laptev, Saeed Amizadeh, and Ian Flint. Generic and scalable framework for automated time-series anomaly detection. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2015.

Alexander Lavin and Subutai Ahmad. Evaluating real-time anomaly detection algorithms: The Numenta anomaly benchmark. In *Proceedings of the IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2015.

Błażej Leporowski, Daniella Tola, Christian Hansen, and Alexandros Iosifidis. AURSAD: Universal robot screwdriving anomaly detection dataset. *arXiv preprint arXiv:2202.03211*, 2022.

Bryan Lim, Sercan Ö. Arık, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.

Mistral AI. Mistral Large 3: Model card, 2025. Mistral AI model card. `https://mistral.ai/news/mistral-large-3/`.

Yuqi Nie, Nam H. Nguyen, Phanwadee Sitharam, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

OpenAI. GPT-5.1 system card, 2025. OpenAI technical report. `https://openai.com/index/gpt-4-1/`.

Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.

Nick Pawlowski, Daniel C. Castro, and Ben Glocker. Deep structural causal models for tractable counterfactual inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. *Elements of Causal Inference*. MIT Press, 2017.

Maxime Peyrard and Robert West. A ladder of causal distances. *arXiv preprint arXiv:2005.02480*, 2020.

Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zhiyuan Zeng, Yujia Huang, Chaojun Xiao, Chi Han, et al. Tool learning with foundation models. *arXiv preprint arXiv:2304.08354*, 2023.

Qwen Team, Alibaba Group. Qwen3 technical report, 2025. arXiv:2505.09388.

Donald B. Rubin. Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology*, 66(5):688–701, 1974.

Lukas Ruff, Robert A. Vandermeulen, Nico Görnitz, Lucas Deecke, Shoaib Ahmed Siddiqui, Alexander Binder, Emmanuel Müller, and Marius Kloft. Deep one-class classification. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2018.

Jakob Runge, Peer Nowack, Marlene Kretschmer, Seth Flaxman, and Dino Sejdinovic. Detecting and quantifying causal associations in large nonlinear time series datasets. *Science Advances*, 5 (11), 2019.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Aarohi Srivastava et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023.

Ya Su, Youjian Zhao, Chenhao Niu, Rong Liu, Wei Sun, and Dan Pei. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. Transformers in time series: A survey. *arXiv preprint arXiv:2202.07125*, 2022.

Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023a.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023b.

Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. MMMU: A massive multi-discipline multimodal understanding and reasoning benchmark for expert AGI. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.

Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2022.

A Answer formats and scoring

FactoryBench emits questions in five answer formats. The four structured formats (single-select MCQ, multi-select MCQ, ranking, tensor) are scored deterministically by per-format rules; free-form answers, used only at Level 4, are scored by an LLM-as-judge voting protocol. Each format is designed to remain hard to answer in the absence of correct reasoning over the time series. Table 5 summarises the raw scoring rules; the rest of this section gives the implementation details together with the chance-correction step that places every format on a comparable scale before aggregation.

Table 5: Answer formats used in FactoryBench and their raw scoring rules. The chance-correction step described in this section is applied on top of these rules before per-level aggregation. Free-form answers (Level 4 only) are scored by an LLM-as-judge voting protocol with three judges and majority vote.

Format	**Description**	**Raw scoring (before chance correction)**
Single-select MCQ	Three or four mutually exclusive options (A–C or A–D); the model selects one.	Exact match: 1 if the parsed letter equals the gold letter, 0 otherwise. Per-item chance level $E = 1/k$ where $k \in \{3, 4\}$ is the option count for that question.
Multi-select MCQ	Four independent statements (A–D) about the outcome of an event or intervention; the model emits a T/F string of length four.	Positional fraction: $1/4$ per slot whose T/F matches the gold; raw score in $\{0, 0.25, 0.5, 0.75, 1\}$. Chance level $E = 1/2$.
Ranking	Four time-series segments or outcomes (A–D) ordered by a specified criterion. Answer is a permutation string (e.g. `DABC`).	Positional fraction: $1/4$ per slot whose letter matches the gold permutation; raw score in $\{0, 0.25, 0.5, 0.75, 1\}$. Chance level $E = 1/4$.
Tensor	One or more scalar values (e.g. expected sensor reading after an intervention). Answer is a list of floats.	Per-scalar three-level piecewise score: 1 if $\lvert\hat{v} - v^\star\rvert \le m$, 0.5 if $m < \lvert\hat{v} - v^\star\rvert \le 2m$, else 0. Per-channel margin calibrated to $m_j = R_j/12$ so the chance level matches MCQ ($E = 1/4$).
Free-form	Open-ended natural-language response, used only for the two Level 4 question types: troubleshooting and optimization.	Score $\in \{0, 0.5, 1\}$. Troubleshooting: 0.5 if the reply names the correct root cause, 1 if it also gives the correct corrective protocol, 0 otherwise. Optimization: 0.5 if the reply correctly identifies the misconfigured parameter, 1 if it also gives the correct corrective protocol, 0 otherwise.

Chance correction. Without correction, formats with different chance baselines are not directly comparable: a model scoring 0.5 on a multi-select item (chance $E = 1/2$) has demonstrated nothing, while a model scoring 0.5 on a single-select item (chance $E = 1/4$) has demonstrated above-chance performance. We therefore transform every raw item score s into a chance-corrected score

$$\tilde{s} \;=\; \max\!\left(0,\; \frac{s - E}{1 - E}\right),$$

where E is the format's chance level. After this transform, $\tilde{s} = 0$ corresponds to pure-chance performance and $\tilde{s} = 1$ to perfect performance for every format. All level- and template-level means reported in the paper are computed on $\tilde{s}$. The simple non-LLM baseline used in Section 5.1 doubles as the empirical audit: after chance correction it should hover near 0 on every format, and we verify this holds before treating any model score as evidence of reasoning.

Single-select MCQ. Raw score $s \in \{0, 1\}$. The number of options k varies by template (between 3 and 4), so the chance level is read off the question payload as $E = 1/k$. The corrected score is $\tilde{s} = \max(0, (k\,s - 1)/(k - 1))$, mapping a correct answer to $\tilde{s} = 1$ and a wrong answer to $\tilde{s} = 0$ regardless of k.

Multi-select MCQ. Raw score $s \in \{0, 0.25, 0.5, 0.75, 1\}$ with chance level $E = 1/2$ (each T/F slot matches the gold with probability $1/2$ under uniform random guessing). The corrected score is

$\tilde{s} = \max(0, 2s - 1)$, mapping $\{0, 0.25, 0.5, 0.75, 1\}$ to $\{0, 0, 0, 0.5, 1\}$. The aggressive flooring is intentional: getting half the slots right under random T/F is the expected outcome of a model that has not read the question.

Ranking. Raw score is the positional-match fraction over a permutation of length four. The expected number of fixed points of a uniformly random permutation is exactly 1 regardless of length, so the chance level is $E = 1/n = 1/4$. The corrected score is $\tilde{s} = \max(0, (4s - 1)/3)$, mapping $\{0, 0.25, 0.5, 0.75, 1\}$ to $\{0, 0, 1/3, 2/3, 1\}$.

Tensor. Each scalar j in a vector answer is scored on a three-level scale: 1 if $|\hat{v}_j - v_j^\star| \leq m_j$, 0.5 if within $2m_j$, 0 otherwise. The raw item score is the average over the n scalars, $s = \frac{1}{n}\sum_j s_j$. The discrete form mirrors MCQ/ranking and still credits near-misses inside $2m_j$. The per-channel margin is calibrated to $m_j = R_j/12$, where R_j is the channel's empirical $[p_2, p_{98}]$ range across the unified episodes (precomputed once); this sets the piecewise scorer's chance expectation to $E = 1/4$, matching single-select MCQ. The corrected item score follows the MCQ form, $\tilde{s} = \max(0, (4s - 1)/3)$. Scalars with a degenerate channel range ($R_j \approx 0$) are dropped from the average.

Free-form. Free-form answers do not admit a meaningful chance baseline: the answer space is unbounded natural language, so there is no uniform random distribution to subtract. We therefore leave free-form scores uncorrected. The lack of correction does not introduce cross-format bias because free-form is used *only* at Level 4, and Level 4 contains no other answer format, so free-form scores are never aggregated alongside structured-format scores at the same level.

The scoring itself is a deterministic rubric on $\{0, 0.5, 1\}$, applied against the gold answer assembled from the FactoryBench knowledge graph. The two Level 4 question types are scored separately:

- **Troubleshooting.** The reply receives 0.5 for naming the correct root cause, 1 for naming both the root cause and the correct corrective protocol, and 0 otherwise.
- **Optimization.** The reply receives 0.5 for correctly identifying the misconfigured parameter, 1 for identifying both the misconfigured parameter and the correct corrective protocol, and 0 otherwise.

Aggregation. Per-question chance-corrected scores $\tilde{s}$ are averaged uniformly within a template, within a level, and overall. Because every $\tilde{s} \in [0, 1]$ has the same operational meaning ("how much of the gap from chance to perfect did the model close on this item"), the per-level numbers in Figure 3 are directly comparable across formats and levels.

B Question template inventory

Table 6 lists the number of question templates currently producing released Q&A items, broken down by answer format. Templates that are defined in the generator but do not yet emit any questions in the released set are excluded.

Table 6: Number of active question templates per level, by answer format. Counts reflect templates that produce released Q&A items in the current dataset.

Level	MC single-select	MC multi-select	Ranking	Tensor / Numerical	Free-form	Total
1 (State)	1	1	0	2	0	4
2 (Intervention)	3	2	2	3	0	10
3 (Counterfactual)	0	2	1	2	0	5
4 (Decision)	0	0	0	0	2	2
Total	4	5	3	7	2	**21**

Add details about how questions are marked. Also, research how to show questions are answerable

B.1 Full template list

Table 7 lists every template defined in the generator. Placeholders in braces ({task}, {anomaly}, {phase}, {signal}, {n}, {event}, {window_length}, {joint_signal}) are filled at generation time from the episode metadata or the relevance specification. The trailing answer-format reminders (*"Answer only with ..."*) are truncated for readability, matching the convention used in Table 16. Note that the *active* subset reported in Table 6 is smaller because some defined templates currently emit no released Q&A items under the active relevance specification.

Table 7: Full list of FactoryBench question templates by causal level. *Single-choice MCQ* requires one letter; *multi-select MCQ* requires a binary string over a fixed option set; *ranking* requires a permutation over k items; *tensor (scalar)* is a single real number accepted within a tolerance band; *tensor (6-vector)* is a JSON array of six real numbers, one per joint axis; *free-form* responses are evaluated by an LLM-as-judge against a reference answer.

ID	Answer format	Template (boilerplate truncated)
Level 1 — State understanding		
L1.1	Tensor (scalar)	The robot is performing a {task} task. We want to isolate the {phase} in the robot's time series. Assuming a fixed window length of {window_length} timesteps, at which timestamp should the window begin?
L1.2	Single-choice MCQ	What anomaly are present in the following time series?
L1.3	Multi-select MCQ	What changed between the two instances of robotic time series data?
L1.5	Ranking	Rank the following robot time series segments from most to least severe anomaly.
L1.6	Single-choice MCQ	What robot does this sensor data originate from?
L1.7	Tensor (scalar)	Given the sensor stream below, what is the expected value of {signal} at T+{n}ms?
Level 2 — Intervention reasoning		
L2.1	Ranking	The sensor stream below is from a robot exhibiting {anomaly}. Rank the signal segments listed in the 'options' field in the order you would expect them to appear as the anomaly manifests.
L2.2	Multi-select MCQ	The sensor stream below is from a robot exhibiting {anomaly}. What would most likely happen next?
L2.3	Multi-select MCQ	The sensor stream below is from a robot exhibiting {anomaly}. Select all statements that apply in the options provided.
L2.4	Tensor (scalar)	The sensor stream below is from a robot exhibiting {anomaly}. What is the expected value of {signal} at T+{n}ms?
L2.5	Tensor (6-vector)	The sensor stream below is from a robot exhibiting {anomaly}. What are the expected values of {joint_signal} at T+{n}ms?
L2.6	Tensor (scalar)	Knowing that the robot suffers from {anomaly} in the given context time series, we want to isolate the {phase} phase. Assuming a fixed window length of {window_length} timesteps, at which timestamp should the window begin?
L2.7	Single-choice MCQ	Given the sensor data from a robot performing the task, determine what anomaly is present?
L2.8	Single-choice MCQ	You are provided with two sensor streams originating from robots accomplishing tasks. What differences between the two given instances of robotic time series data (if any) do you notice?
L2.9	Ranking	Rank the following robot time series segments from most to least severe anomaly.
L2.10	Single-choice MCQ	Knowing that the robot suffers from {anomaly} in the given context time series, what robot does this sensor data originate from?
Level 3 — Counterfactual reasoning		

continued on next page

Table 7 continued from previous page

ID	Answer format	Template (boilerplate truncated)
L3.1	Ranking	Given the baseline sensor stream below and the counterfactual scenario where {event}, rank the signal segments listed in the ‘options’ field in the order you would expect them to appear as the event manifests.
L3.2	Multi-select MCQ	Given the counterfactual scenario where {event}, what would most likely happen next?
L3.3	Multi-select MCQ	Given the counterfactual scenario where {event}, select all statements that would apply.
L3.4	Tensor (scalar)	In the counterfactual scenario where {event}, what would be the expected value of {signal} at T+{n}ms?
L3.5	Tensor (6-vector)	In the counterfactual scenario where {event}, what would be the expected values of {joint_signal} at T+{n}ms?
Level 4 — Decision support		
L4.1	Free-form	Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the most likely root cause and describe the steps you would take to fix it.
L4.2	Free-form	An engineer wants to increase the effectiveness and accuracy of this machine. Based on the sensor stream below, what operational changes or parameter adjustments could help achieve this? Do not justify your answer, simply indicate steps to take (or specify if nothing can be done).
L4.3	Ranking	The 4 sensor streams below each correspond to a complete task execution padded to a common length. Consider only the task completion portion of each stream (up to the end of the final task phase) when assessing duration. Rank them from shortest to longest task duration.
L4.4	Ranking	The 4 sensor streams below each correspond to a complete task execution padded to a common length. Consider only the task completion portion of each stream (up to the end of the final task phase) when assessing energy consumption. Rank them from most to least energy-efficient.

Prompt structure and knowledge-graph augmentation. Models under evaluation receive a single user message; no system role is set on the evaluated model. The message is assembled at prompt-build time from four ordered blocks:

```
<machine_sentence>
<acronym_mapping>
<time_series>
Question: <question_text>
Here are the options:
<options>
```

The leading `<machine_sentence>` is filled at prompt-build time from the FactoryBench knowledge graph—specifically the machine and gripper tables, keyed on the dataset that the question is grounded in. This guarantees that identical question templates are grounded in the correct hardware description regardless of which underlying dataset the episode comes from. For a UR3e-grounded question on FactoryWave, the resulting line expands to:

> The following sensor data comes from Universal Robots UR3e, a collaborative robot (cobot) from the E-series with 3 kg payload, 6 degrees of freedom, controlled via UR PolyScope (teach pendant), UrScript, typically used for light assembly tasks, automated workbench scenarios. It is equipped with a 2FG14 Finger Gripper OnRobot.

The augmentation is conditional: identification templates such as L1.6 (“What robot does this sensor data originate from?”) and L2.10 suppress the machine sentence so the model cannot read the answer off the prompt header, and templates that probe gripping behaviour without revealing tooling can selectively suppress only the gripper clause. The acronym-mapping block decodes the time-series header into long-form channel names and is included whenever the question carries one.

For Level-4 free-form responses, model outputs are scored by an LLM-as-judge using a separate system prompt that is fixed across templates and is not augmented from the knowledge graph:

```
You are an expert evaluator for a robotics sensor-data Q&A benchmark.
Your task is to score a model's free-form answer against a reference
answer.

Scoring criteria (0-10):
  10 - Answer is semantically equivalent to the reference: correct
       direction of change, correct signal, correct magnitude range,
       correct time window.
   7 - Answer captures the main trend and signal correctly but is
       imprecise on magnitude or timing.
   4 - Answer mentions the right signal but the described behaviour
       is partially incorrect or vague.
   1 - Answer is on-topic but mostly incorrect or misleading.
   0 - Answer is completely wrong, irrelevant, or refuses to answer.

Respond ONLY with a JSON object in this exact format (no extra text):
{"score": <integer 0-10>, "reason": "<one sentence justification>"}
```

C Noise-substitution check

To probe whether the model scores in Figure 3 reflect actual reading of the time-series context or merely priors over question templates and option positions, we constructed a noise-substituted counterpart of the dataset and re-ran every model against it.

Construction. For each of the 400 generated Q&A pairs (100 per level), we duplicate the item and rewrite only the numerical time-series content. For each numeric *float* feature we preserve the original first value and substitute every subsequent value with the cumulative sum of independent Gaussian increments:

$$v'_0 = v_0, \qquad v'_{t+1} = v'_t + \mathcal{N}(0, \sigma^2), \qquad \sigma = 0.5,$$

where σ is shared across all features. The same substitution is applied to time-series chunks embedded in option strings (Level 1 severity-ranking, Level 2/3 chunk-ranking).

Expected behaviour. A model that scores by genuinely reading the time series should drop sharply on the noise-substituted variant, because the substitution destroys the signal-trajectory information that the gold answer hinges on. A model that scores by template-level priors (e.g. guessing the most plausible MC option given the question wording) or by option-position bias should be roughly invariant, since the question text and options are unchanged.

Result. Table 8 reports the side-by-side mean rubric scores. Three patterns emerge.

1. On Level 1 every model is essentially flat or improves slightly under substitution. This is consistent with L1 templates being substantially answerable from the question wording, the option set, and the still-present discrete annotations of task phase and fault label. It also flags L1 as the level where overlap between LLM scores and a random/regression baseline is most expected.
2. On Level 2 and Level 3 every model drops, often substantially: DeepSeek and Mistral lose 9–11 points on L3, and gpt-5.1 loses 7.7 on L2. The L3 drops are particularly informative because L3 templates ask counterfactual "what would have happened" questions where the answer hinges on the actual signal trajectory rather than on a discrete annotation.
3. On Level 4, gpt-5.1's 7.6% original score collapses to 0.5% under noise substitution — a $15\times$ drop. Inspection of the per-item rubric notes shows that gpt-5.1's L4 mass on the

original data came almost entirely from correctly emitting "no anomaly is present" on truly nominal episodes; once the time series is replaced by Gaussian noise, the same nominal episodes look anomalous to the model and it now hallucinates faults on them, losing its only mode of scoring. The other three models score ≤ 0.005 in either condition and have no comparable mode to lose.

Table 8: Noise-substitution check: measured mean scores (%) on the original FactoryBench Q&A pairs (Orig) vs. the noise-substituted variant (Sub) constructed by replacing every numerical time-series value with cumulative Gaussian increments seeded at the original first value ($\sigma = 0.5$). Integer-valued schema fields are preserved. Questions, options, and gold answers are identical between the two runs. Δ = Sub − Orig; large negative values are evidence that the model was reading the time series rather than the question alone.

	L1			L2			L3			L4		
Method	Orig	Sub	Δ	Orig	Sub	Δ	Orig	Sub	Δ	Orig	Sub	Δ
Claude Haiku 4.5	19.0	19.5	+0.5	25.5	26.7	+1.2	6.3	4.4	-1.9	0.5	0.0	-0.5
DeepSeek V3.1	16.5	15.0	-1.5	23.7	19.5	-4.2	24.7	15.7	-9.0	3.5	4.0	+0.5
gpt-5.1	24.0	24.0	0.0	28.7	21.0	-7.7	22.0	16.0	-6.0	7.6	0.5	-7.1
Mistral-Large-3	22.0	25.0	+3.0	22.7	21.8	-0.9	26.7	15.8	-10.9	0.0	0.0	0.0

D FactoryBench-Lite: distillation methodology and validation

E Details of the FactoryWave dataset

FactoryWave was collected on two one-armed platforms: the **Universal Robots UR3** (cobot; OnRobot Screwdriver and two-finger gripper variants) and the **KUKA KR10** (industrial arm). Each episode corresponds to one complete task cycle; episodes are recorded under three conditions (normal, fault, and counterfactual), described below. Because state interpretation rather than policy quality is the object of study, motion is generated by a deterministic scripted controller. Every sample carries extensive labelling metadata, and the full native-rate sensor stream is logged without online feature selection.

E.1 Tasks and phase structure

Three canonical tasks are represented in the dataset: Pick-and-Place, Screwing, and Peg-in-Hole. All three are executed on both robots, using matched scripted controllers and task layouts so that cross-embodiment comparisons hold the task fixed. We structure each task as a deterministic sequence of phases (Tables 9, 10, 11); the phase index is written as a discrete label at every timestep, providing an interpretable segmentation that downstream models can condition on or that can be used to localize questions to a specific stage of the cycle. Within this fixed phase skeleton, we randomize as much of the episode as the task permits so that a question template that targets a given phase does not accidentally learn a fixed pose, timing, or trajectory. Pick and place locations are sampled uniformly from predefined workspace regions; phase-specific joint velocities, accelerations, and end-effector (EEF) rotations are drawn from bounded per-phase ranges; and approach angles to contact events are perturbed within the tolerances of the controller. This forces the Q&A pairs generated from FactoryWave to reflect phase-level reasoning rather than memorisation of a single scripted trajectory, which in turn makes them generalizable to a wide variety of trajectories and variation found in real recordings.

check eef is common to use

E.2 Episode metadata

Alongside the sensor stream, every episode carries a fixed set of metadata fields describing the robot, task, condition, payload, and end-effector configuration (Table 12). These fields are constant within an episode and are used both as filters during question generation (e.g. to restrict a template to a specific payload regime) and as provenance in the released Q&A pairs. This includes fault-specific fields (such as injection timestamp or physical offset).

Table 9: Task phases for Pick-and-Place.

Phase index	Phase name	Description
0	`above_pick`	EEF moves to a waypoint directly above the object
1	`descend`	EEF lowers toward the object
2	`settle`	Brief hold to let physics settle before grasping
3	`close`	Gripper closes around the object
4	`lift`	EEF lifts the grasped object off the surface
5	`move_xy`	Lateral transfer toward the bin
6	`lower`	EEF lowers into the bin
7	`open`	Gripper opens, object released
8	`retract`	EEF moves away from the bin
9	`return`	EEF returns to home configuration

Table 10: Task phases for Screwing.

Phase index	Phase name	Description
0	`approach`	EEF moves to a pre-contact waypoint above the fastener
1	`descend`	EEF lowers to engage the fastener
2	`screw`	Continuous downward force + wrist rotation to thread
3	`disengage`	Gripper opens / tool lifts off
4	`retract`	EEF returns to a safe height
5	`descend`	EEF lowers to engage the fastener (loosening pass)
6	`loosen`	Loosen the screw
7	`engage`	Gripper closes on the retrieved screw
8	`retract`	EEF returns to home position

Table 11: Task phases for Peg-in-Hole.

Phase index	Phase name	Description
0	`above_hole`	EEF moves above the hole and aligns
1	`insert`	EEF pushes peg downward into the hole
2	`disengage`	Gripper opens
3	`above_hole`	EEF rises back to the waypoint above the hole
4	`retract`	EEF returns to home position
5	`above_object`	EEF moves above the object and aligns with it
6	`engage`	Gripper closes
7	`above_object`	EEF rises back to the waypoint above the object
8	`retract`	EEF returns to home position

Table 12: Default per-episode metadata fields.

Field	Description
`episode_id`	Unique identifier for the episode
`robot_model`	Robot model (UR3, KUKA KR10)
`task`	Task (pick-and-place, screwing, peg-in-hole)
`condition`	Episode condition (normal, fault, counterfactual)
`fault_id`	Fault index for fault episodes; null otherwise
`weight_of_box`	Mass of the transported object in kg (pick-and-place only)
`shape_of_box`	Shape/dimensions of the transported object (pick-and-place only)
`position_of_box`	Initial XYZ position of the object on the table, where measurable
`gripper_model`	Gripper model and type (vacuum, two-finger, screwdriver)
`payload_mass_configured`	Payload mass configured at the controller (kg)
`payload_cog_configured`	Payload CoG offset configured at the controller (x, y, z in mm)
`tcp_offset_configured`	TCP position offset configured at the controller (x, y, z in mm)
`tcp_orientation_configured`	TCP orientation configured at the controller (rx, ry, rz in radians)

E.3 Fault taxonomy

FactoryWave includes 27 distinct fault types (Table 13). Each fault is paired with a plain-language description of the underlying physical or control-level mechanism and the injection procedure used to induce it during data collection. The last three columns indicate which tasks include the fault in the collected dataset; faults that apply to all three tasks (e.g. collisions with a shared workspace object, misconfigurations of the shared controller) are recorded under each applicable task with task-specific signal imprints.

add images of experiments

Table 13: Fault catalogue. Columns *PP*, *Scr*, *PiH* indicate whether the fault is included in the Pick-and-Place, Screwing, and Peg-in-Hole recordings.

Fault	Explanation	Injection	PP	Scr	PiH
Damaged screw thread	Screw thread physically damaged, preventing proper engagement during tightening.	Pre-damaged the screw thread with sandpaper.	×	✓	×
Missing screw	Tightening is attempted with no screw present.	Removed the screw from screwdriver before the cycle.	×	✓	×
Damaged plate thread	The threaded hole in the plate is damaged, preventing engagement.	Pre-damaged the plate hole with a metal screwdriver.	×	✓	×
Loosening phase	A counterclockwise loosening rotation replaces tightening; a normal phase with a distinct signal. It is counted here as an anomaly to match the fault schema of the AURSAD dataset Leporowski et al. [2022].	Reversed the rotation direction in the controller program to execute a counterclockwise loosening pass in place of tightening.	×	✓	×
Gripper activation failure	The vacuum gripper fails to activate and never picks up the box.	Disabled the gripper activation command in the robot program so the pick cycle completes without a grasp.	✓	×	×
Gripper release during motion	The gripper releases the payload mid-trajectory, causing an abrupt payload loss.	Triggered a programmatic gripper-release command at a scripted timestep mid-trajectory.	✓	×	×
Additional axis payload	A dead weight attached to one link increases inertia and gravity loading on all joints.	Bolted calibrated weights of varying mass to one robot link.	✓	✓	×
Collision with foam object	Contact with a soft foam block produces a brief TCP force spike without a protective stop.	Placed a foam cube directly in the programmed TCP trajectory.	✓	✓	✓
Unexpected payload weight	The transported box has a weight that deviates from the nominal payload configuration.	Swapped the nominal box for a known heavier or lighter one. This is done in a counterfactual context, ie. switching weights and asking about alternate weight scenario.	✓	×	×
Invalid gripping position	Timing error: the gripper closes after the arm has begun lifting, gripping the object off-centre.	Inserted a brief delay in the controller program so gripper closure lags the lift motion.	✓	×	×

Table 13 (cont.)

Fault	Explanation	Injection	PP	Scr	PiH
Unstable mounting platform	Base instability introduces low-frequency vibrations into the arm.	Placed 8 layers of towels under the base plate.	✓	×	×
Joint position limit violation	A joint moves outside its configured soft or hard position limits, triggering a safety stop.	Random waypoint slightly beyond the configured soft joint limit.	✓	×	×
TCP frame misconfiguration	TCP frame or mounting orientation misconfigured at the controller; gravity torques deviate.	Entered an incorrect TCP offset or mounting angle in the controller installation settings.	✓	✓	✓
Payload weight misconfiguration	Payload mass misconfigured while a tool or workpiece is physically attached.	Left the configured payload weight at multiple wrong mass configurations while a task-dependant gripper remained mounted.	✓	✓	×
External arm disturbance	A continuous external force pulls or pushes the TCP during motion.	Anchored an elastic rope between the bench frame and the tool flange.	✓	✓	✓
Mild payload CoG misconfiguration	Payload CoG specified at an incorrect offset from the tool flange; gravity compensation is wrong.	Entered a CoG offset in the payload configuration that differs from the physical tool CoG.	✓	✓	✓
Collision with hanging cable	A loose cable drags along a robot link or the tool flange during motion.	Suspended a loose cable across the trajectory at arm height.	✓	✓	✓
Collision with cardboard object	A cardboard carton in the trajectory provides moderate resistance; may trigger a protective stop.	Placed a free-standing or braced cardboard box in the programmed trajectory.	✓	✓	✓
Collision with rigid object	A rigid plastic block in the TCP path triggers an immediate protective stop.	Clamped a rigid plastic block in the programmed TCP path. Unlike the other collisions, this fault consistently causes safety stops.	✓	✓	✓
Peg insertion misalignment	Peg approaches the hole at an angular or lateral offset and jams against the rim.	Added a recorded lateral offset at the insertion approach waypoint.	×	×	✓
Hole obstruction	Foreign object or debris in the hole prevents full insertion.	Placed a small object (metal screw) inside the hole before insertion.	×	×	✓
Incorrect insertion depth	Insertion terminates at an incorrect Z depth due to a misconfigured waypoint.	Shifted the insertion endpoint waypoint Z by recorded offset from nominal.	×	×	✓
Peg surface contamination	Contamination or roughness on the peg raises insertion friction and produces stick-slip.	Applied dry chalk powder to the peg surface.	×	×	✓
Fixture displacement	Insertion fixture shifted laterally, breaking the match between programmed approach and actual hole.	Physically shifted the hole fixture by recorded offset without updating the robot program.	×	×	✓

Table 13 (cont.)

Fault	Explanation	Injection	PP	Scr	PiH
Self-collision	Arm configuration causes a link to contact the robot body or mounting fixture, triggering a protective stop.	Random waypoint that forces a self-colliding configuration or offset the mounting fixture into the trajectory.	✓	×	×
Missing box	Box absent from the pick position; the gripper closes on empty air.	Removed the box from the pick position before the episode started.	✓	×	×
Missing peg	Arm descends into the hole empty-handed; insertion produces no contact force.	Removed the peg from the gripper before the episode started.	×	×	✓

E.4 Counterfactual episodes

Counterfactual pairs are constructed for the subset of faults in Table 13 whose injection moment can be controlled at a specific timestep rather than being present throughout the episode. For each such fault, the initial conditions of the physical setup (object identity and pose, payload configuration, controller settings) are held as constant as the platform allows, one baseline execution is recorded without any injection, and several fault executions are then recorded under the same initial conditions with the fault applied at a shared sampled timestep. Because the physical system is not perfectly reproducible, the pre-injection sensor trajectories of the baseline and the fault candidates diverge slightly; we retain, as the counterfactual partner of the baseline, the fault candidate whose pre-injection window is closest to the baseline under the signature kernel MMD, and discard the others. This yields an approximate counterfactual pair in which the early history is as matched as the real platform permits, while the post-injection divergence isolates the effect of the intervention. The approximation error introduced by this procedure, relative to the exact counterfactuals available in simulation, motivates the sim2real analysis in Section 5.3.

E.5 Signal schema

The UR3 is recorded at 125 Hz, yielding 136 per-sample signals grouped by semantic role (Table 14): controller *setpoints*, actual *effort / feedback* readings from joints and the TCP, *context / health* signals covering joint temperatures, voltages, and internal controller state, a small block of controller *status / mode* indicators, and digital/analog *I/O* lines. A handful of episode-level provenance fields (episode identifier, robot type, task, fault label, payload mass, recording date) are appended per episode but are constant within an episode.

Table 14: UR3 signal groups recorded at 125 Hz (excluding episode-level provenance columns).

Group	# signals	Contents
Setpoint (intent)	42	Target joint position, velocity, acceleration, current, torque; target TCP pose and speed.
Effort / feedback	48	Actual joint position, velocity, current, current-as-torque, joint control output; actual TCP pose, speed, and force/torque.
Context / health	33	Joint temperatures, joint voltages, joint modes, tool accelerometer, raw F/T wrench, main/robot voltage and current, momentum, speed scaling, execution time.
Status / mode	7	Timestamp, robot mode, robot status, safety mode, safety status bits, runtime state, configured payload mass.
I/O	6	Digital inputs/outputs, two analog inputs, two analog outputs.
Total	**136**	

The KUKA KR10 is controlled through the KRC4 controller. We stream 44 per-sample signals at 83 Hz, grouped by semantic role in Table 15 using the same partition as the UR3 schema. A handful of signals that the UR3 exposes natively are unavailable on KSS 8.3, specifically joint velocities, commanded TCP pose, joint voltage, the tool-mounted accelerometer, and joint-level control outputs; TCP force/torque is also absent because no force sensor is fitted.

Table 15: KUKA KR10 signal groups recorded at 83 Hz via RSI/KRL (excluding episode-level provenance columns).

Group	# signals	Contents
Setpoint (intent)	6	Commanded joint positions (degrees).
Effort / feedback	24	Actual joint positions (degrees), actual TCP pose (mm / degrees), per-axis motor current (% of max), and per-axis motor torque (Nm).
Context / health	11	Per-axis motor temperature (Kelvin), Cartesian acceleration on X, Y, Z and absolute magnitude (m/s^2), and speed override (%).
Status / mode	1	Controller process state (FREE / ACTIVE / STOP / END).
I/O	2	Digital inputs bitmask and digital outputs bitmask.
Total	**44**	

F Example question-answer pairs and question templates

Table 16 presents seventeen representative Q&A pairs spanning every causal level and every answer format the generator produces (single-select MCQ, multi-select MCQ, tensor, ranking, free-form). For readability we omit the underlying sensor context (tens to hundreds of time-series rows), truncate verbose boilerplate such as "Answer only with . . .", and abbreviate long option strings.

Table 16: Representative Q&A pairs from FactoryBench spanning all four causal levels and every answer format.

Level	Answer format	Prompt, options, and reference answer
L1	Tensor	**Q:** The robot is performing a screwing task. We want to isolate the screwing phase in the robot's time series. Assuming a fixed window length of 10 timesteps, at which timestamp should the window begin? **A:** `14` (accepted within ± 3)
L1	Single-select MCQ	**Q:** What changed between the two instances of robotic time series data? **Options:** **a.** different anomalous states (exactly one anomalous, or both but distinct) **b.** different tasks **c.** different robots **d.** same task, different phases **A:** `a`
L1	Single-select MCQ	**Q:** What robot does this sensor data originate from? **Options:** **a.** Agile Robots Yu 5 Industrial **b.** KUKA KR 10 R1100-2 **c.** Universal Robots UR3 **A:** `c`
L1	Tensor	**Q:** Given the sensor stream below, what is the expected value of the position of joint 2 at T+6 ms? **A:** `102.7777`
L2	Tensor	**Q:** The sensor stream below is from a robot exhibiting a collision with a cardboard object. What is the expected value of the velocity of joint 3 at T+68 ms? **A:** `-0.272044` (accepted within ± 2.42)

continued on next page

Table 16 continued from previous page

Level	Answer format	Prompt, options, and reference answer
L2	Tensor	**Q:** The sensor stream below is from a robot exhibiting a collision with a cardboard object. What are the expected values of joint positions at T+506 ms? **A:** `[17.884, -115.897, 121.049, -95.595, -90.258, 293.860]`
L2	Tensor	**Q:** Knowing that the robot suffers from a hole obstruction, we want to isolate the "rise back above the object" phase. Assuming a fixed window length of 16 timesteps, at which timestamp should the window begin? **A:** `202` (accepted within ± 3)
L2	Single-select MCQ	**Q:** Given the sensor data, determine what anomaly is present. **Options:** **a.** sustained command/measurement deviation **b.** no anomaly **c.** measured joint speeds drop abruptly with a TCP force spike (typical collision signature) **d.** isolated unrealistic force-sensor spike inconsistent with motion **A:** `c`
L3	Ranking	**Q:** Given the baseline stream and the counterfactual scenario where a collision foam object occurs at timestep 7366 ms, rank the four labeled signal segments in the order they would appear as the event manifests. **A:** `abdc`
L3	Multi-select MCQ	**Q:** Given the counterfactual scenario where a collision hanging cable occurs at timestep 8363 ms, select all statements that would apply. **Options:** **a.** motor current rises $\geq 33\%$ while speed stays $\leq 10\%$ of baseline (stall-like) **b.** contact-force spike $\geq 36\%$ above baseline **c.** tracking-error rise $\geq 34\%$ above pre-event mean **d.** at least one joint speed drops sharply ($\geq 31\%$) **A:** `FFFT`
L3	Tensor	**Q:** In the counterfactual scenario where a collision foam object occurs at timestep 7659 ms, what would be the expected value of the position of joint 2 at T+100 ms? **A:** `72.144939` (accepted within ± 2.77)
L4	Free form	**Q:** Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the most likely root cause and describe the steps you would take to fix it. **A:** No anomalous behavior detected; the machine is operating normally; no remediation is required.
L4	Free form	**Q:** Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the most likely root cause and describe the steps you would take to fix it. **A:** **a.** Inspect the workspace and remove any soft obstructions from the programmed trajectory. **b.** If a protective stop occurred, acknowledge it in the robot controller and verify arm posture before resuming. **c.** Reduce speed if repeated false-positive collision detections occur.

continued on next page

Table 16 continued from previous page

Level	Answer format	Prompt, options, and reference answer
L4	Free form	**Q:** Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the most likely root cause and describe the steps you would take to fix it. **A:** **a.** Stop the robot and update the payload configuration to include the additional axis weight. **b.** Set the correct CoG offset for the modified arm in the installation settings. **c.** Reduce motion accelerations and speeds to account for increased effective inertia. **d.** Run a slow-speed test cycle to verify dynamic loads stay within rated limits before resuming.
L4	Free form	**Q:** An engineer wants to increase the effectiveness and accuracy of this machine. Based on the sensor stream below, what operational changes or parameter adjustments could help achieve this? **A:** The payload mass is set to 0.0 kg but the actual payload weighs 1.5 kg. Update the payload mass in the installation settings to 1.5 kg.
L4	Free form	**Q:** An engineer wants to increase the effectiveness and accuracy of this machine. Based on the sensor stream below, what operational changes or parameter adjustments could help achieve this? **A:** The TCP offset is misconfigured at [0, 0.3, 55] instead of the correct [0, 0, 55]. Update the TCP position offset in the installation settings to [0, 0, 55].
L4	Free form	**Q:** An engineer wants to increase the effectiveness and accuracy of this machine. Based on the sensor stream below, what operational changes or parameter adjustments could help achieve this? **A:** The payload center of gravity is set to [25, 0, 20] but the correct value is [0, 0, 20]. Update the CoG offset in the installation settings to [0, 0, 20].

G Simulation setup

After Karim finishes the simulations, we fill this section up. Describe the simulation pipeline used for sim2real analysis: physics engine and version, robot URDFs, controller stack, supported tasks and fault applicators, randomization strategy, logging frequency and signal coverage, and how simulated episodes are aligned with the FactoryWave schema.

H Causal schema (SCE): full details

This appendix gives the long-form definition of the Setpoint–Context–Effort + Feedback (SCE) schema used to organise every time-series channel in FactoryBench, together with the residual-based fault definition that the schema enables. The motivation is summarised in Section 3.2; here we cover the per-group channel breakdown, the residual formalism, and the per-robot calibration that makes faults computable rather than imputed.

The schema groups every recorded channel into one of three causal roles. **Setpoint** captures the controller's commanded target: per-axis position, velocity, and acceleration setpoints emitted by the trajectory generator. **Context** captures physical and configured conditions affecting behaviour, split into *static* context (episode-level metadata such as payload mass, payload centre of gravity, TCP offset, gripper model, and box / peg geometry) and *dynamic* context (per-sample channels such as joint temperatures, supply voltages, controller mode, and safety mode). **Effort + Feedback** captures the machine's measured response: motor currents and torques (effort), measured joint positions and TCP pose (feedback), tool accelerometer, and acoustic emission where available. The exact per-robot channel mapping appears in the signal-group tables in this appendix (UR3 and KUKA KR10).

This split enables a single operational definition of a fault that applies across machines and tasks. Under healthy operation, the measured Effort + Feedback response $\mathbf{y}_t$ is by definition a function of the Setpoint command $\mathbf{S}$ and the Context $\mathbf{C}_t$:

$$\mathbf{y}_t = f(\mathbf{S}, \mathbf{C}_t),$$

where f represents the nominal plant dynamics (inverse-dynamics-plus-controller transfer) calibrated per robot. Faults manifest as structured residuals from this expected baseline: $\mathbf{y}_t - f(\mathbf{S}, \mathbf{C}_t)$ forms a clear, fault-specific spatiotemporal pattern. Because every episode supplies the paired $(\mathbf{S}, \mathbf{C}_t, \mathbf{y}_t)$ streams, this residual is computable directly from the data, giving ground truth that does not have to be hand-imputed. Figure 4 summarises the causal relationships between the three groups.

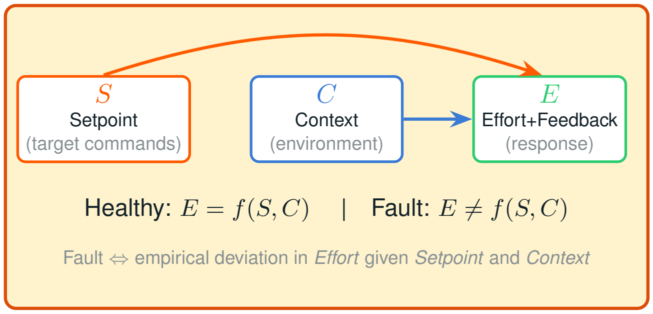

Figure 4: Physical causal schema. Setpoint commands S and contextual variables C jointly determine the measured Effort + Feedback response E under healthy operation. Faults manifest as residuals $E - f(S, C)$ from this nominal mapping.